

Distances :-

(i) Euclidean (L2 norm) :

$$a \in \mathbb{R}^D, b \in \mathbb{R}^D$$

$$\begin{aligned} d_2(a, b) &= \|a - b\|_2 = \sqrt{\sum_{i=1}^D (a_i - b_i)^2} = \sqrt{(a-b)^T (a-b)} \\ &= \sqrt{a^T a + b^T b - 2a^T b} \end{aligned}$$

(ii) Weighted Euclidean Distance :-

$$a \in \mathbb{R}^D, b \in \mathbb{R}^D$$

$$d_w(a, b) = \sqrt{\sum_{i=1}^D w_i (a_i - b_i)^2} = \sqrt{(a-b)^T w (a-b)}$$

↓
DxD diagonal
matrix with
weights w_i on
it's diagonal.
weights may be
known or even
learned from data.

Note: If w is DxD symmetric matrix then it is called Mahalanobis distance.

(iii) Manhattan Distance | L1 norm | Taxicab Norm :

$$d_1(a, b) = \|a - b\|_1 = \sum_{i=1}^D |a_i - b_i|$$

(iv) L-∞ distance :-

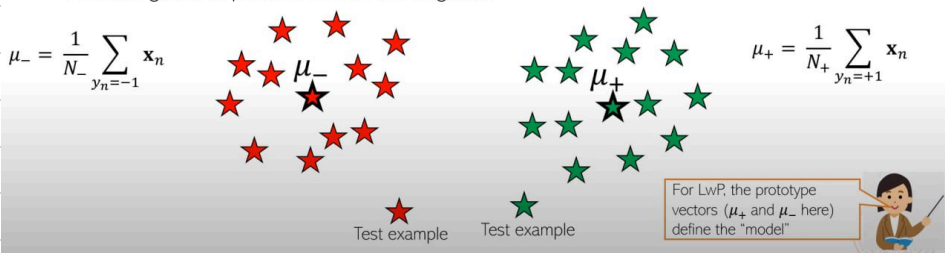
Max of abs-value of element wise difference between two vectors.

Learning with Prototypes :-

- Represent each class by prototype vector.
- **Class prototype** : The "mean" or "average" of inputs from that class.
- Predict label of each test input based on it's distances from the class prototypes.
- Predicted label will be the class that is closest to the test input.
- How we compute distances can affect the accuracy of the model.

Ex: Learning with Prototypes (LwP): An Illustration

- Suppose the task is binary classification (two classes assumed pos and neg)
- Training data: N labelled examples $\{(\mathbf{x}_n, y_n)\}_{n=1}^N$, $\mathbf{x}_n \in \mathbb{R}^D$, $y_n \in \{-1, +1\}$
 - Assume N_+ example from positive class, N_- examples from negative class
 - Assume green is positive and red is negative

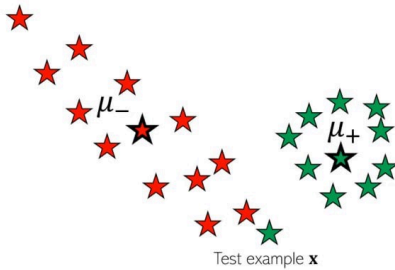


Note: For multiclass classification with K classes we will consider K prototypes.

LwP Drawbacks:-

LwP: Some Failure Cases

- Here is a case where LwP with Euclidean distance may not work well



- In general, if classes are not equisized and spherical, LwP with Euclidean distance will usually not work well (but improvements possible; will discuss later)

solⁿ:- i) Feature Scaling ii) Different distance metric
(Mahalanobis distance)

Parametric Models:- Can throw away training data after computing prototypes and just need to keep the model parameters for the test time in such "parametric" models.

Ex: LwP

Nearest Neighbors :-

steps:-

- i) Compute the distance of the test point from all training inputs.
 - ii) Sort the distance to find the "nearest" input(s) in training data.
 - iii) Predict the label using majority on avg labels of these inputs.
- Unlike LWP which does prototypes based comparison, nearest neighbors method looks at the labels of individual training inputs to make prediction.
 - LWP can only work for classification problem whereas Nearest Neighbors applicable to both classification and regression.

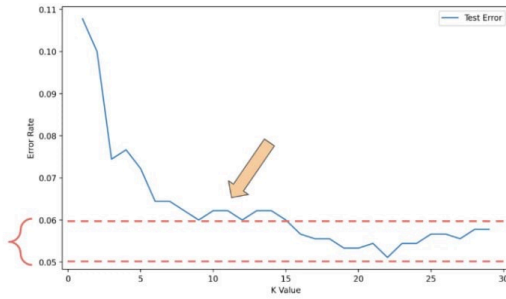
How to choose K ?

→ i) Heuristic :- $\sqrt{\text{\# observations}}$

ii) Elbow Method :-

$$\text{Error} = 1 - \text{Accuracy}$$

- Elbow method:

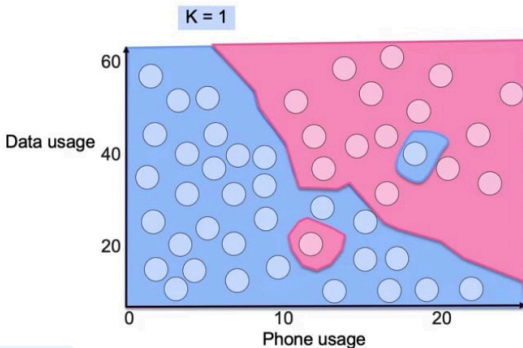


after some K value we are not reducing error that much.
we are increasing complexity of the model.

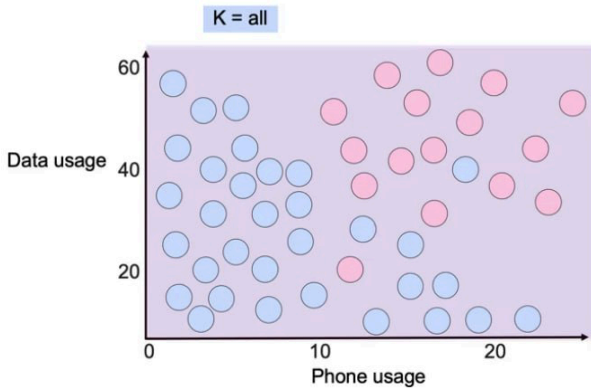
iii) Cross Validation :-

- Cross validation only takes into account the K value with the lowest error rate across multiple folds.
- This could result in a more complex model (higher value of K).
- Consider the context of the problem to decide if larger K values are an issue.

Overfitting and Underfitting :-



→ overfitting



→ under fitting

→ model will always predict the majority class.

Pros:

- Simple to implement (does not require estimation)
- Adapts well as new training data
- Easy to interpret

Cons:

- Slow to predict because many distance calculations
- Does not generate insight into data generating process (no model)
- Can require lots of memory if data set is large (or as it grows)
- When there are many predictors, KNN accuracy can break down due to curse of dimensionality → In higher dimension distance metrics are not reliable.

- outliers.

- Imbalanced Dataset

- Black box model : Inference can't be done only prediction.

Comparison with Linear Regression and Logistic Regression :

Linear Regression

- Fitting involves minimizing cost function (slow)
- Model has few parameters (memory efficient)
- Prediction involves calculation (fast)

K Nearest Neighbors

- Fitting involves storing training data (fast)
- Model has many parameters (memory intensive)
- Prediction involves finding closest neighbors (slow)

- **KNN Considerations:**

- Scaling for Distance
 - Features could have vastly different value ranges!



wnt Feature 1.
(vastly
different)

wnt Feature 2.
(Almost same)

- **KNN Considerations:**

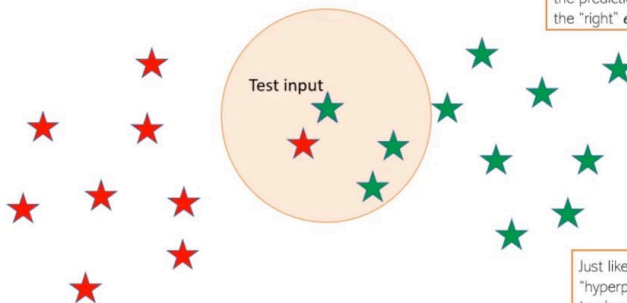
- Scaling is necessary for KNN.

Some Variants of KNN :

ϵ -Ball Nearest Neighbors (ϵ -NN)

11

- Rather than looking at a fixed number K of neighbors, can look inside a ball of a given radius ϵ , around the test input



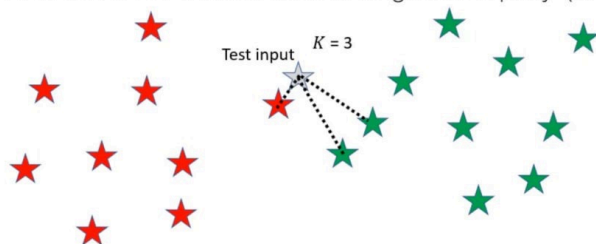
So changing ϵ may change the prediction. How to pick the "right" ϵ value?

Just like K , ϵ is also a "hyperparameter". One way to choose it is using "cross-validation" (will see shortly)

Distance-weighted KNN and ϵ -NN

12

- The standard KNN and ϵ -NN treat all nearest neighbors equally (all vote equally)



- An improvement: When voting, give more importance to closer training inputs

Unweighted KNN prediction: $\frac{1}{3} \text{ (red star)} + \frac{1}{3} \text{ (green star)} + \frac{1}{3} \text{ (green star)} = \text{green star}$

Weighted KNN prediction: $\frac{3}{5} \text{ (red star)} + \frac{1}{5} \text{ (green star)} + \frac{1}{5} \text{ (green star)} = \text{red star}$



Note:- For multi-class, simply use the same majority rule like binary classification case.

- KNN is a non-parametric method.

Time and Space Complexity:-

$$T = O(nd), \quad n: \text{no. of training points.} \\ d: \text{no. of features.}$$

For each observation loop will run d times for distance calculation. (let $\underline{x} \in \mathbb{R}^d$)

$$S = O(nd)$$



KD-Tree

- Improvement over KNN

- $T = O(d \log n)$

K-D tree example

- Building a K-D tree from training data:
 - pick random dimension, find median, split data, repeat
- Find NNs for new point (7,4)
 - find region containing (7,4)
 - compare to all points in region

(1,9), (2,3), (4,1), (3,7), (5,4), (6,8), (7,2), (8,8), (7,9), (9,6)

